



哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY



分布式系统核心 (I)

RPC机制

王宏志

哈尔滨工业大学

计算学部 海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

目录

Contents

- 1 **RPC 基本模型与工作机制**
- 2 **RPC 失败模型与容错机制**
- 3 **RPC 调用语义与系统设计**

目录

Contents

- 1 **RPC 基本模型与工作机制**
- 2 **RPC 失败模型与容错机制**
- 3 **RPC 调用语义与系统设计**

分布式系统中的通信实体

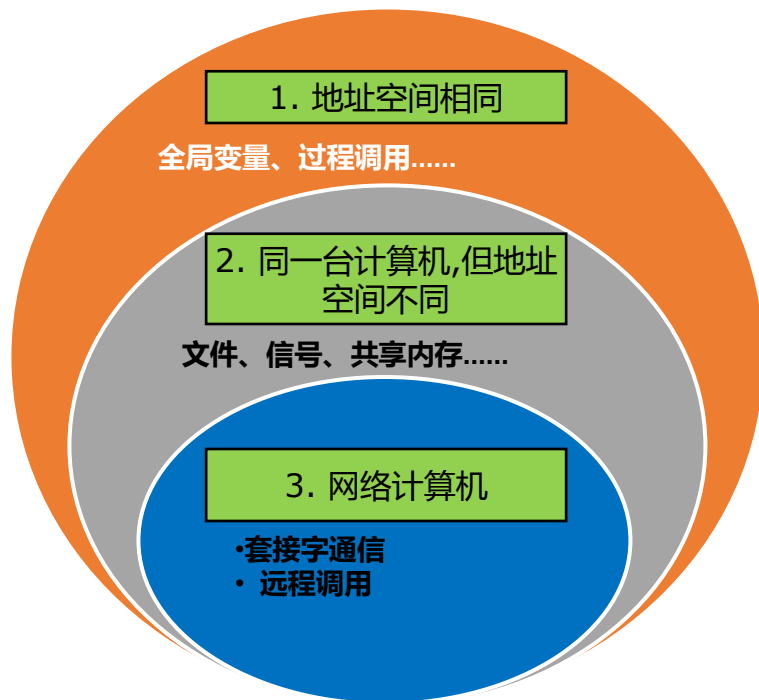
- 分布式系统中的通信实体可分为两种:
 - 系统导向实体
 - 过程
 - 线程
 - 节点
 - 以问题为导向的实体
 - 对象(在基于对象编程的方法中)

分布式系统中的实体如何通信？

- 通信范式描述并分类了一组实体可以交互和交换数据的方法

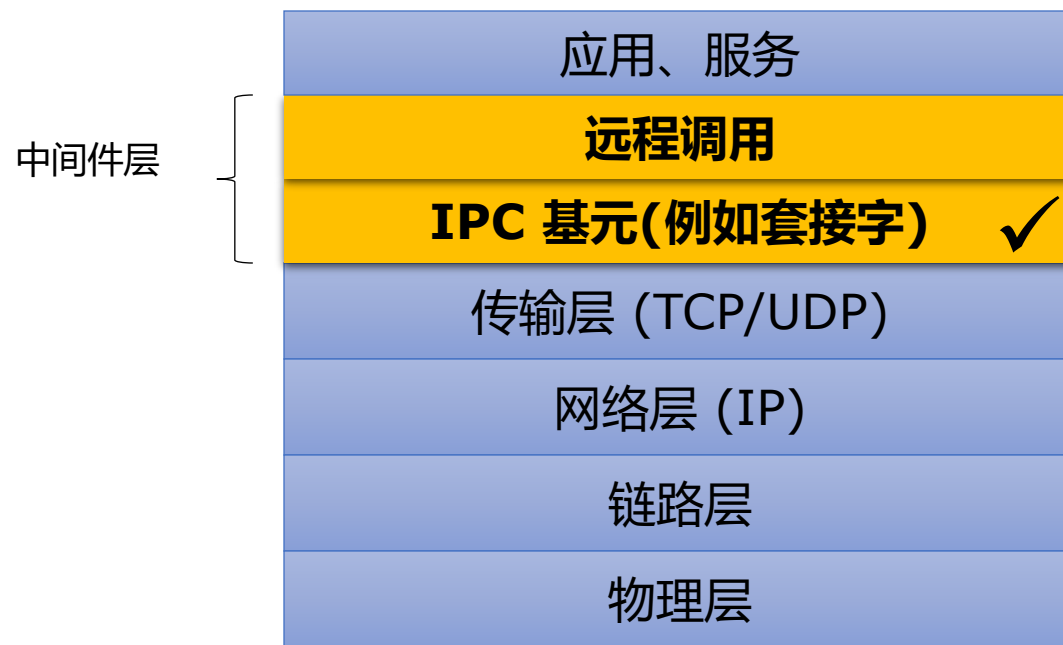
传播范式的分类

- 通信范式根据实体所在,可分为三种。 如果实体运行在:



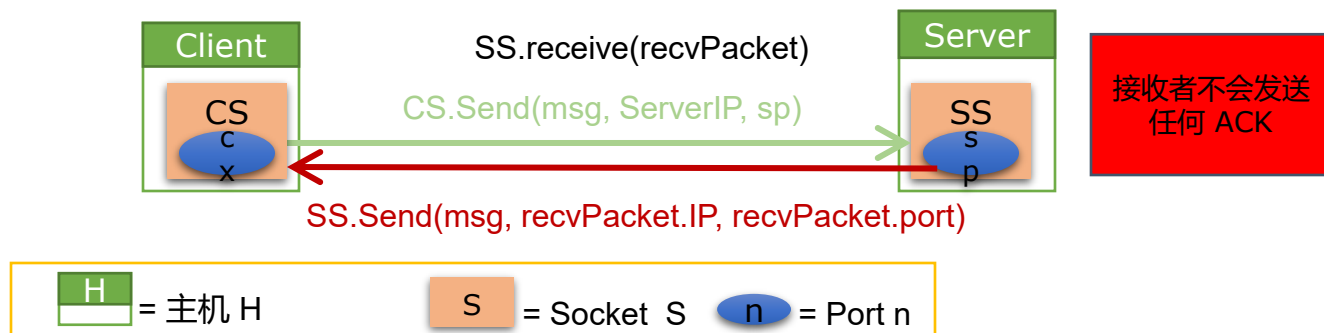
今天,我们将研究驻留在联网计算机上的实体如何利用套接字通信和远程调用在分布式系统中进行通信

中间件层



UDP 套接字

- UDP 提供无连接通信,没有确认或消息重传
- 通信机制:
 - Server 在已知端口 sp 处打开 UDP 套接字 SS,
 - Socket SS 等待接收请求
 - 客户端在随机端口 cx 处打开 UDP 套接字 CS
 - 客户端套接字 CS 向 ServerIP 和端口 sp 发送消息
 - 服务器套接字 SS 可以将数据发回 CS

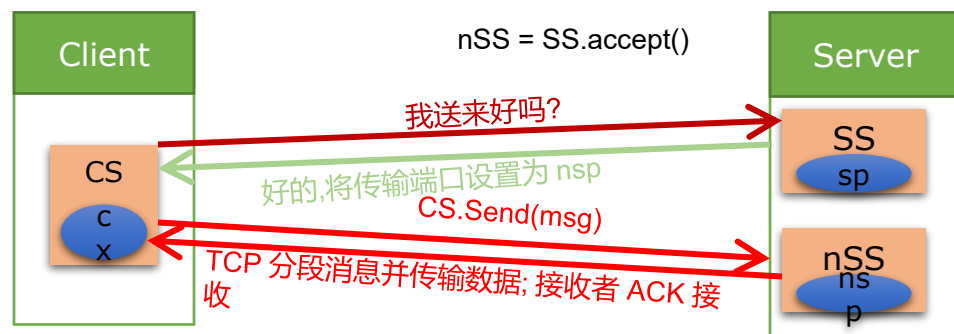


UDP- 设计注意事项

- 发送者必须明确地将长消息碎片化为更小的块,然后才能传输
 - 建议传输最大大小为 548 字节
- 消息可能会失序传递
 - 如有必要,程序员必须重新排序数据包
- 沟通不可靠
 - 消息可能是由于校验和错误或路由器上的缓冲区溢出而丢弃的
- 接收者应分配一个足够大的缓冲区,以容纳发送者的信息
- 否则,消息将被截断

TCP 套接字

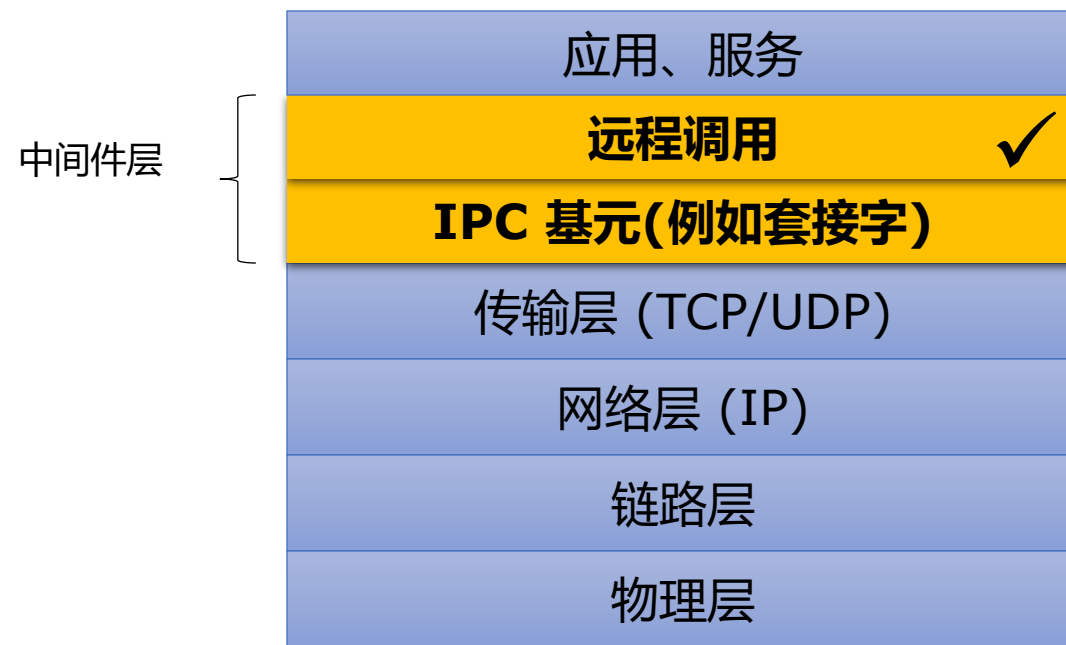
- TCP 提供按顺序交付、可靠性和拥塞控制
-
- 通信机制:
 - Server 在已知端口 sp 处打开 TCP 服务器套接字 SS
 - 服务器等待接收请求(使用接受调用)
 - 客户端在随机端口 cx 处打开 TCP 套接字 CS
 - CS 向 ServerIP 和端口 sp 发起连接启动消息
 - 服务器套接字 SS 在随机端口 nsp 上为客户端分配一个新的套接字 NSS
 - CS 可以将数据发送到 NSS



TCP 的主要优点

- TCP 确保消息按顺序传递
-
- 应用程序可以发送任何大小的消息
-
- TCP 通过使用确认和重传来确保可靠的通信
-
- TCP 的拥塞控制调节发送者速率,从而防止网络过载

中间件层

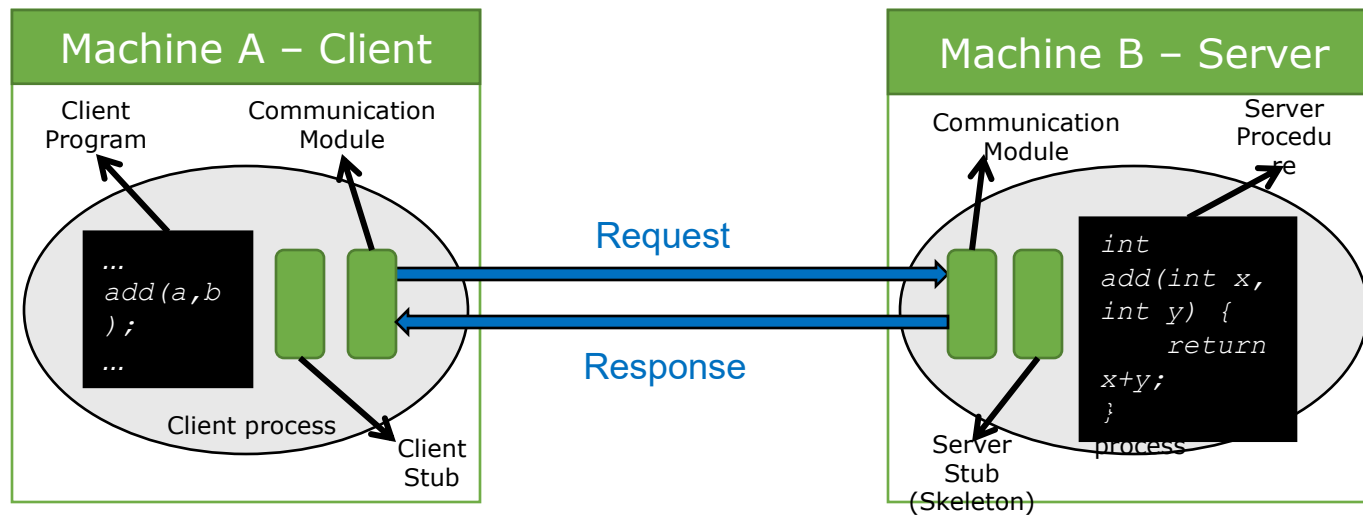


远程调用

- 远程调用使实体能够调用通常在另一台计算机上执行的过程,而无需程序员显式编码通信的细节
 - 底层中间件将处理原始通信
 - 程序员可以透明地与远程实体通信
- 我们将研究两种远程调用:
 - a.Remote Procedure Calls (RPC)
 - b.Remote Method Invocation (RMI)

远程过程调用 (RPC)

- RPC 使发送者能够使用简单的过程调用与接收者通信
 - 程序员看不到任何通信或消息传递
- 基本 RPC 方法:



客户端存根

- 客户端存根：
 - 由用户代码作为本地过程调用
 - 将参数打包（或序列化或打包）成一个请求包（例如，request-pkt）
 - 调用客户端传输例程（例如，makerpc(request-pkt, &reply-pkt)）
 - 将reply-pkt解包（或反序列化或解打包）到输出参数
 - 返回到用户代码

服务器存根

- 服务器存根：
 - 在服务器端传输例程（例如，`getrequest()`）返回后调用
 - 反序列化参数，解复用操作码，并调用本地服务器代码
 - 序列化参数，调用服务器端传输例程（例如，`sendresponse()`），并返回到服务器循环
- 例如，典型的服务器主循环：

```
while (1) {  
    get-request (&p);    /* blocking call */  
    execute-request (p); /* demux based on opcode */  
}
```


RPC 的挑战

- 通过封装处理传递参数
 - 程序参数和结果必须以比特的形式通过网络传输
- 数据表示
 - 数据表示必须统一
 - 发送机和接收机的架构可能不同
- 失败独立性
 - 客户端和服务端可能独立失败

- 通过封送处理传递参数
 - 程序参数和结果必须以比特的形式通过网络传输
- 数据表示
 - 数据表示必须统一
 - 发送机和接收机的架构可能不同
- 失败独立性
 - 客户端和服务端可能独立失败

通过封送处理传递参数

- 将参数打包进将要通过网络传输的消息称为参数序列化。
- 在通过网络传输之前，必须对过程参数和结果进行序列化。
- 可以传递两种类型的参数。：
 1. Value parameters
 2. Reference parameters

1. 传递值参数

- 值参数具有有关变量的完整信息,可以直接编码到消息中
 - E.g., integer, float, character
- 值通过按值调用传递
 - 被调用程序所做的更改不反映在调用程序中

2. 传递引用参数

- 在 RPC 中传递值参数等引用参数会导致结果不正确,原因有二:
 - a. 服务器上引用参数无效
 - 引用参数仅在客户端的地址空间内有效
 - 解决方法:通过复制所引用的数据来传递引用参数
 - b. 引用参数的更改不会反映在客户端
 - 解决方案:“复制/恢复”数据
 - 复制参数所引用的数据
 - 将服务器上的值复制回客户端

RPC 的挑战

- 通过封装处理传递参数
 - 程序参数和结果必须以比特的形式通过网络传输
- 数据表示
 - 数据表示必须统一
 - 发送机和接收机的架构可能不同
- 失败独立性
 - 客户端和服务端可能独立失败

- DS中的计算机通常具有不同的架构和操作系统
 - 数据类型的大小不同
 - 例如,长数据类型在 32 位 Unix 中是 4 字节,在 64 位 Unix 系统中是 8 字节
 - 存储数据的格式不同
 - 例如,Intel 以小端格式存储数据,而 SPARC 以大端格式存储数据
- 客户端和服务端必须就消息中简单数据的表示方式达成一致
 - 例如,整数、字符和浮点等数据类型的格式和大小

RPC 的挑战

- 通过封装处理传递参数
 - 程序参数和结果必须以比特的形式通过网络传输
- 数据表示
 - 数据表示必须统一
 - 发送机和接收机的架构可能不同
- 失败独立性
 - 客户端和服务端可能独立失败

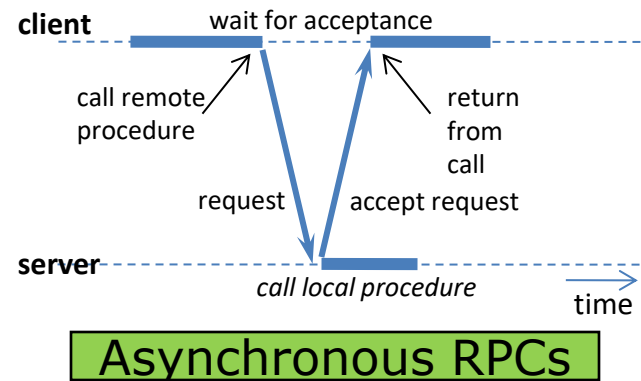
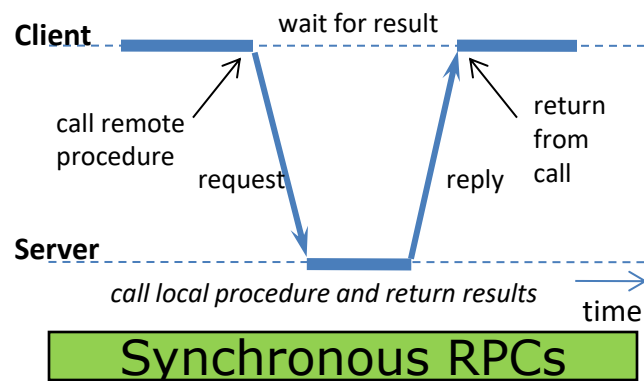
失败独立性

- 在本地情况下,客户端和服务端生死并存
- 在远程案例中,客户看到新的故障类型(详见下讲)
 - 网络故障
 - 服务器崩溃
 - 服务器进程崩溃
- 因此,处理故障的代码必须更彻底(而且本质上更复杂)

- 远程过程调用可以:
 - 同步
 - 异步(或延迟同步)

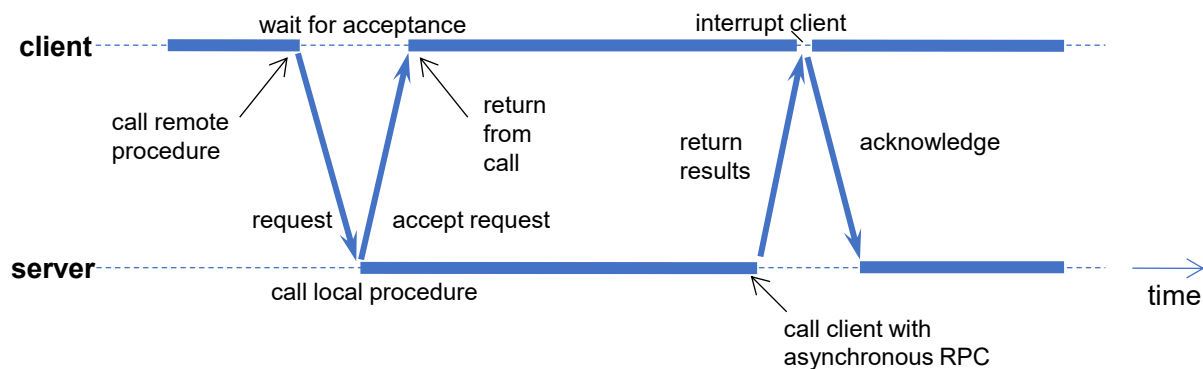
同步与异步 RPC

- 具有严格请求-回复的 RPC 会阻止客户端,直到服务器返回
 - 阻止浪费客户的资源
- 如果客户端不需要服务器的结果,则使用异步 RPC
 - 服务器立即将 ACK 发送回客户端
 - 客户端在服务器发出 ACK 后继续执行



延迟同步 RPC

- 当客户端想要结果,但不想在调用完成之前被阻止时,异步 RPC 也很有用
- 客户端使用延迟同步 RPC
- 单个请求响应 RPC 分为两个 RPC
 - 首先,客户端在服务器上触发异步 RPC
 - 其次,完成后,服务器回调客户端以交付结果



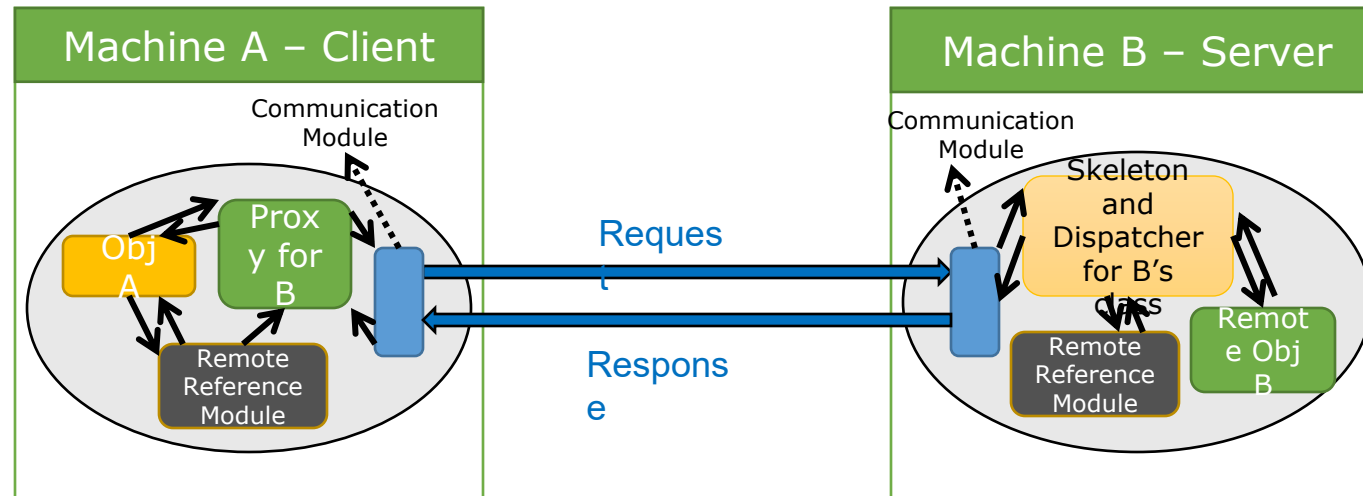
远程方法调用 (RMI)

- RMI 与 RPC 相似,但在分布式对象的世界中
 - 程序员可以充分利用面向对象编程的表达能力
 - RMI 不仅允许传递值参数,还允许传递对象引用
- 在 RMI 中,调用对象可以调用潜在远程对象上的方法

远程对象和支持模块

- 在 RMI 中,其方法可以远程调用的对象称为"远程对象"
 - 远程对象实现远程接口
- 在任何方法调用期间,系统必须解析该方法是在本地对象上还是在远程对象上调用
 - 本地调用应在本地对象上调用
 - 远程调用应通过远程方法调用
 - 远程引用模块负责在本地和远程对象引用之间进行转换

RMI 控制流

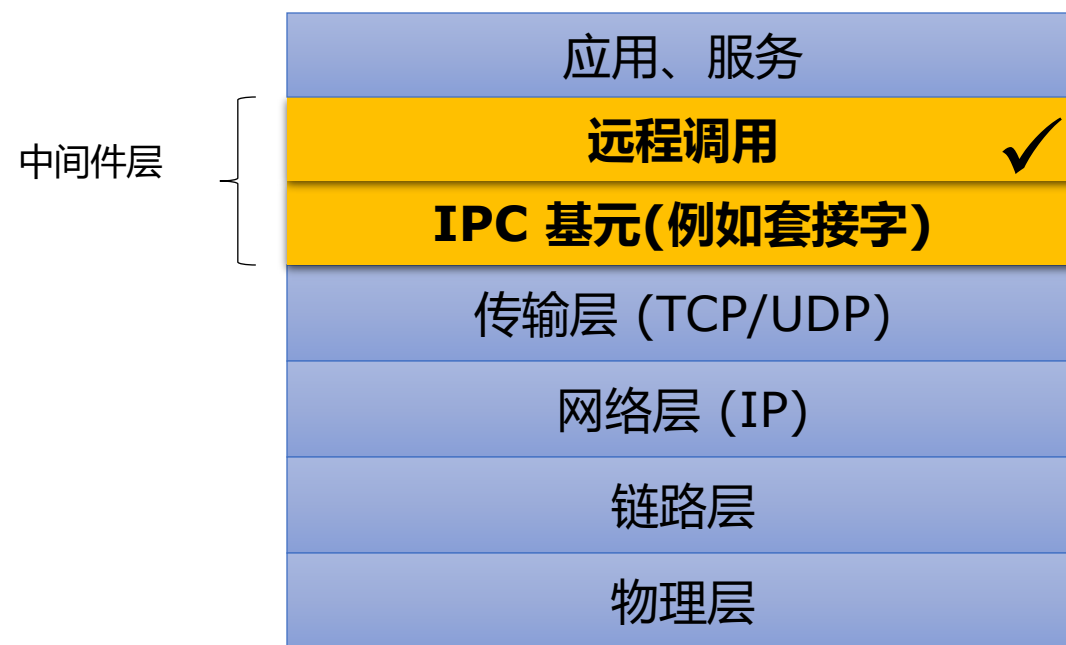


目录

Contents

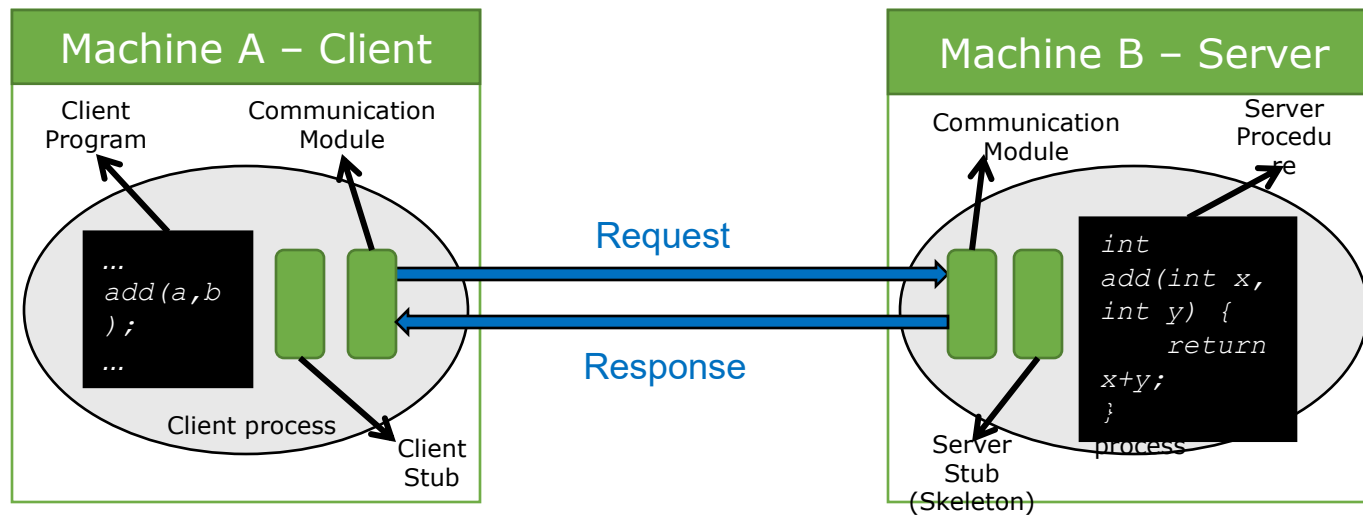
- 1 RPC 基本模型与工作机制
- 2 RPC 失败模型与容错机制**
- 3 RPC 调用语义与系统设计

中间件层

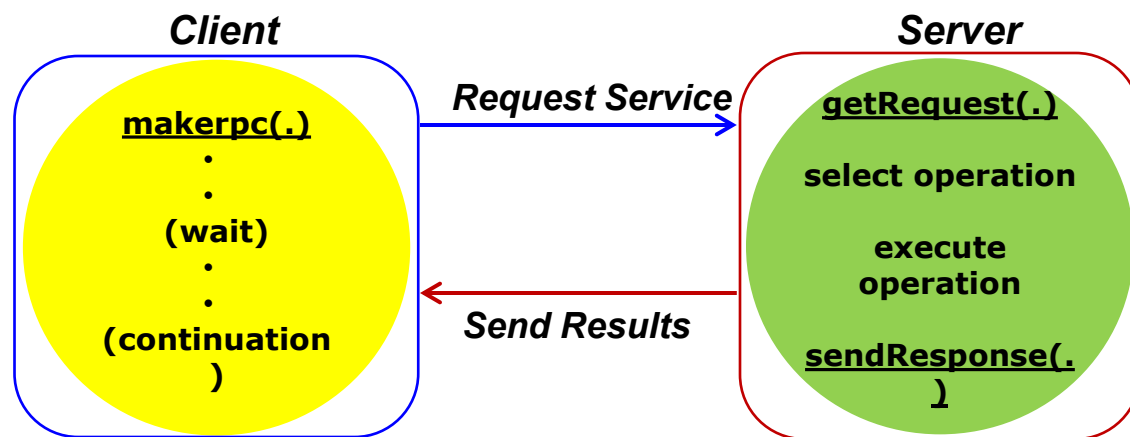


远程过程调用 (RPC)

- RPC 使发送者能够使用简单的过程调用与接收者通信
 - 程序员看不到任何通信或消息传递
- 基本 RPC 方法:



- RPC通信模块（或传输）主要基于一组通信原语，*makerpc(.)*，*getRequest(.)*，和*sendResponse(.)*



故障类型

■ RPC 系统可能会遭受各种类型的故障

故障类型	描述
• 崩溃失败	• 服务器停止,但工作正常,直到停止
• 遗漏失败	• 服务器不能响应传入的请求
• 接收遗漏	• 服务器无法接收传入消息
• 发送遗漏	• 服务器发送消息失败
• 定时失败	• 服务器的响应在指定的时间间隔之外
• 响应失败	• 服务器的响应不正确
• 值失败	• 对答的价值是错误的
• 状态转换失败	• 服务器偏离了正确的控制流
• 拜占庭故障	• 服务器可以在任意时间产生任意响应

超时机制

- 为了允许丢失请求或回复消息的情况,make_rpc(.) 可以使用[超时机制](#)
- make_rpc(.) 在超时后可以做什么,有多种选择:
 - 要么立即返回,并指示客户端请求失败
 - 或反复重传请求,直到收到回复或假定服务器失败为止
- 如何选择超时值?
 - 充其量,用实证/理论统计
 - 最坏的情况是没有好的价值

幂等操作

- 在请求消息被重新传输的情况下，服务器可能会收到不止一次
- 这可能会导致同一请求多次执行操作
- 警告: 并非每个操作都可以执行不止一次，每次都能获得相同的结果！
- 可以以相同效果重复执行的操作被称为幂等操作

重复过滤

- 为避免操作出现问题,服务器应:
 - 识别来自"同一"客户端的连续消息
 - 单调递增序数可用
 - 过滤掉重复的
- 收到"重复"请求后,服务器可以:
 - 或者重新执行操作并回复
 - Possible only for idempotent operations
 - 或避免重新执行操作通过保留其输出在非易失性历史(或日志)文件中
 - 可能需要交易语义(稍后会详细介绍)

实施选择

- RPC 传输可以以不同的方式实现,以提供不同的交付保证。 主要选择是:
 1. 重试请求服务(*client side*):控制是否重新传输请求服务,直到收到应答或假定服务器失败为止
 2. 重复过滤(*server side*):控制何时使用重传,以及是否在服务器上过滤掉重复的请求
 3. 保留结果(*server side*):控制是否保留结果消息的历史记录,以便使丢失的回复能够重新传输,而无需在服务器上重新执行操作

目录

Contents

- 1 RPC 基本模型与工作机制
- 2 RPC 失败模型与容错机制
- 3 **RPC 调用语义与系统设计**

RPC 调用语义

- 度量的组合导致 RPC 可靠性的多种可能语义

容错度量			调用语义(与远程过程有关)
重传请求消息	重复过滤	重新执行程序或重新传输回复	
No	N/A	N/A	也许
Yes	No	重新执行程序	至少一次
Yes	Yes	重传回复	最多一次
yes	yes	Retransmit Reply	At-most-once

理想情况下,我们想要一个恰好一次的语义!

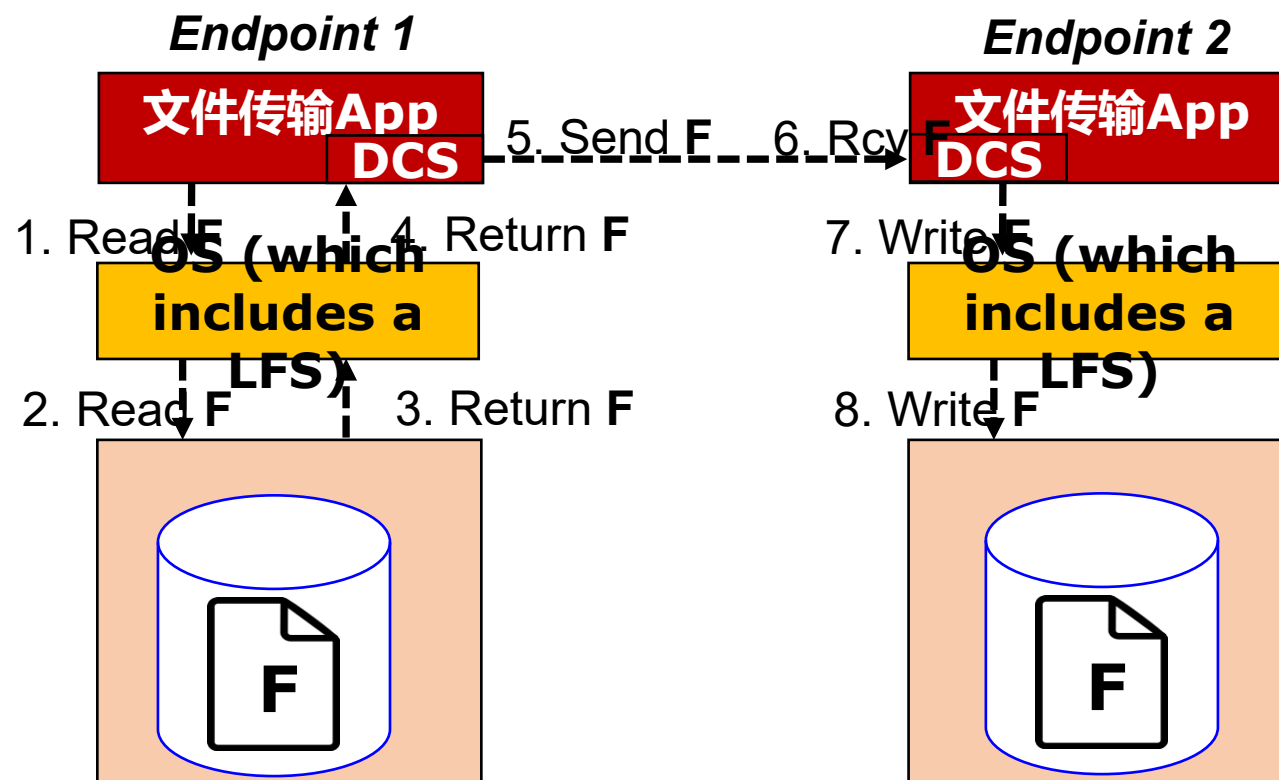
中间件层



RPC over UDP or TCP

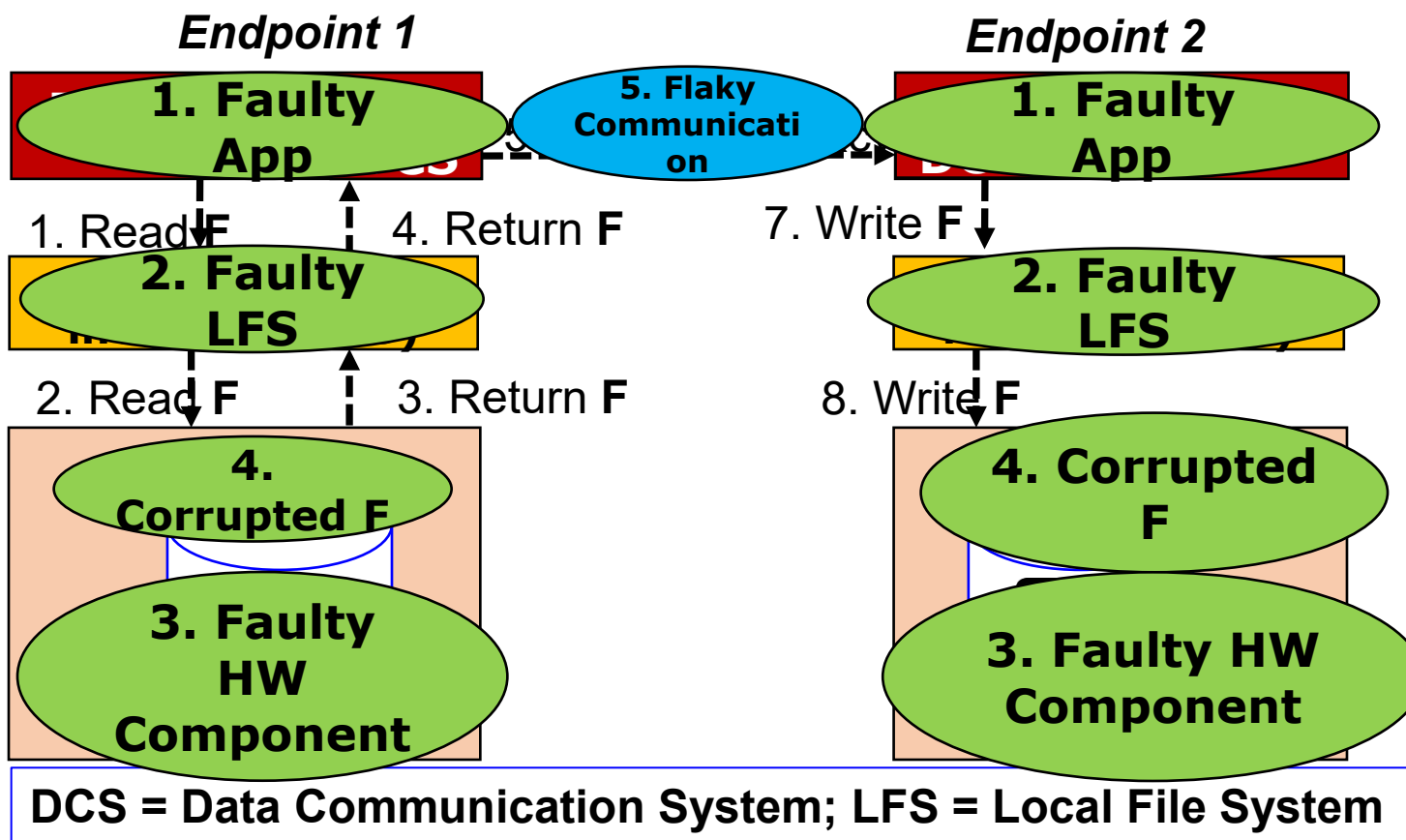
- 如果 RPC建立在 UDP 之上
 - 重传应/可由RPC处理
- 如果RPC建立在TCP之上
 - 重传将由 TCP 处理
 - RPC 内是否仍需要采取容错措施?
 - 是的,读 Saltzer 等的《系统设计中的端到端论证》/al.

谨慎文件传输：流程



DCS = Data Communication System; LFS = Local File System

小心文件传输:可能的威胁



小心文件传输:端到端检查和重试

- 端点1使用F存储校验和 C_A
- 在端点2写入F之后, 它再次从磁盘读取它, 计算校验和 C_B , 并将其发送回端点1
- 端点 1 比较 C_A 和 C_B
 - 如果 $C_A = C_B$,提交文件传输
 - 否则,重试文件传输

小心文件传输:端到端检查和重试

- 重试多少次?
 - 如果很少失败,通常为1
 - 重试 3 次,可能表明系统的某些部分需要修复
- 如果数据通信系统使用 TCP 怎么办?
 - 只有威胁 5(例如,由于通信不稳定而导致的数据包丢失)被消除
 - 如果故障是由通信系统引起的,则重试的次数会减少
 - 更多的控制交通,但只有 F 的缺失部分需要重新装运
 - 文件传输应用程序仍需要应用端到端的可靠性措施!